Chapter 1 Experimental Results

Experiments were conducted to verify the proposed delay management method within an OpenAirInterface (OAI) nFAPI End-to-End environment. To ensure the statistical reliability of the evaluations, each reported throughput value represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial).

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduler algorithm, the experimental scenarios are structured as follows: Section 1.1 outlines the software, hardware environment, and the automated evaluation framework. Section 1.2 validates the Exponentially Weighted Moving Average (EWMA) mechanism for the PNF arrival offset (Δt_{arrive}). Section 1.3 benchmarks the performance in terms of peak throughput and MAC Round-Trip Time (RTT) against static baselines. Section 1.4 evaluates the stability of the proposed method under varying levels of network congestion. Finally, an rApp demonstration video is provided to confirm the end-to-end functional deployment.

1.1 Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

To execute these scenarios rigorously, an automated evaluation frame-

Table 1.1: Testbed Node Configuration

Unit	**Specification**
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

Table 1.2: Wireless System Parameters

Parameter	**Setting**
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

work was developed as an rApp, as illustrated in Figure 1.1. This rApp automates the entire end-to-end experimental process, including test execution, data collection, and performance chart generation. A key capability of this rApp is the automated configuration of network latency simulation via Linux Traffic Control (TC) at both the MAC feedback link ($T_5 - T_4$) and the fronthaul segment ($T_2 - T_1$). This controlled latency injection is essential to validate the split-architecture stability under non-deterministic latency.

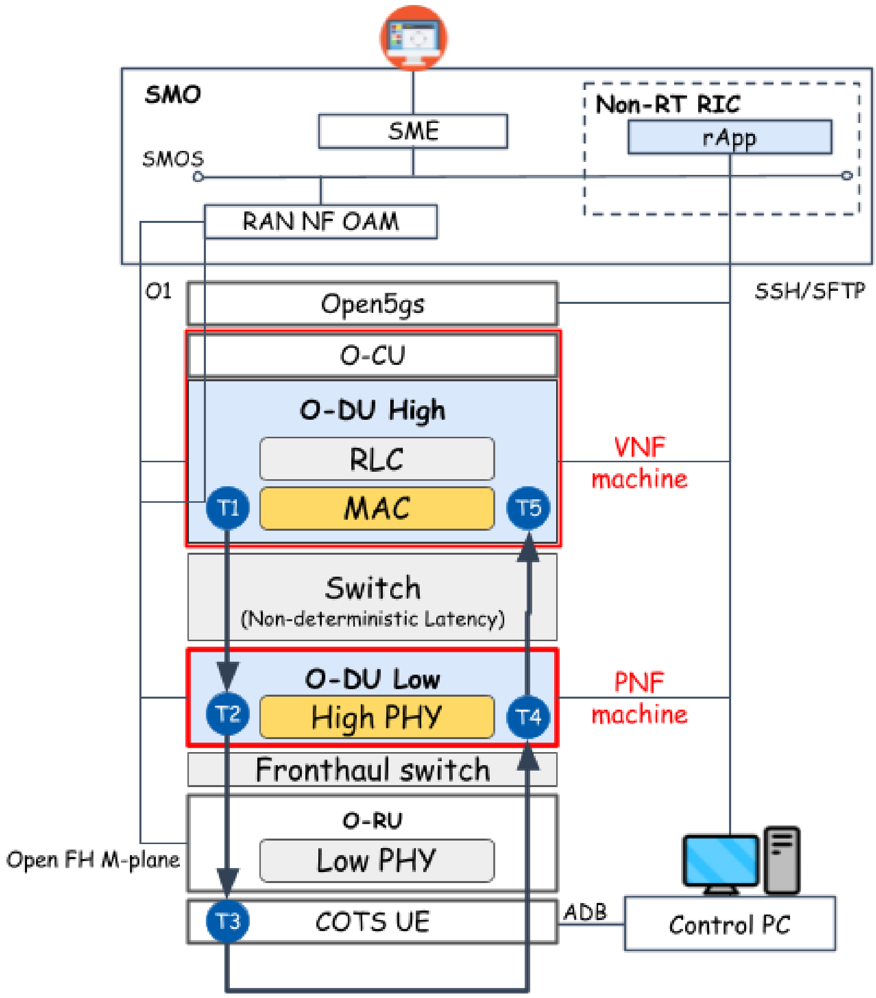
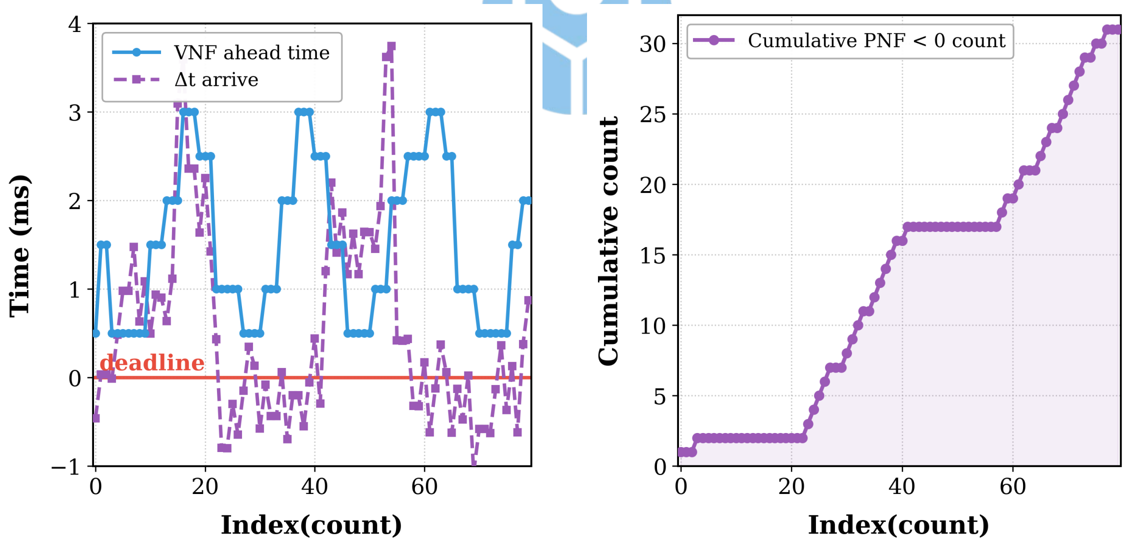


Figure 1.1: Architecture of the rApp automated evaluation framework.

1.2 Scenario I: Validation of EWMA for PNF Arrival Offset (Δt_{arrive})

This scenario validates the effectiveness of applying an Exponentially Weighted Moving Average (EWMA) to smooth the PNF arrival offset (Δt_{arrive}). The raw timing offset received from the PNF can exhibit high variance due to network jitter and processing delays. Figure 1.2 illustrates the impact of EWMA processing. Without EWMA (Figure 1.2a), the arrival offset fluctuates significantly, which could lead to unstable scheduling decisions. By applying EWMA (Figure 1.2b), the high-frequency jitter is effectively filtered out, providing a stable baseline for the timing adjustment algorithm.



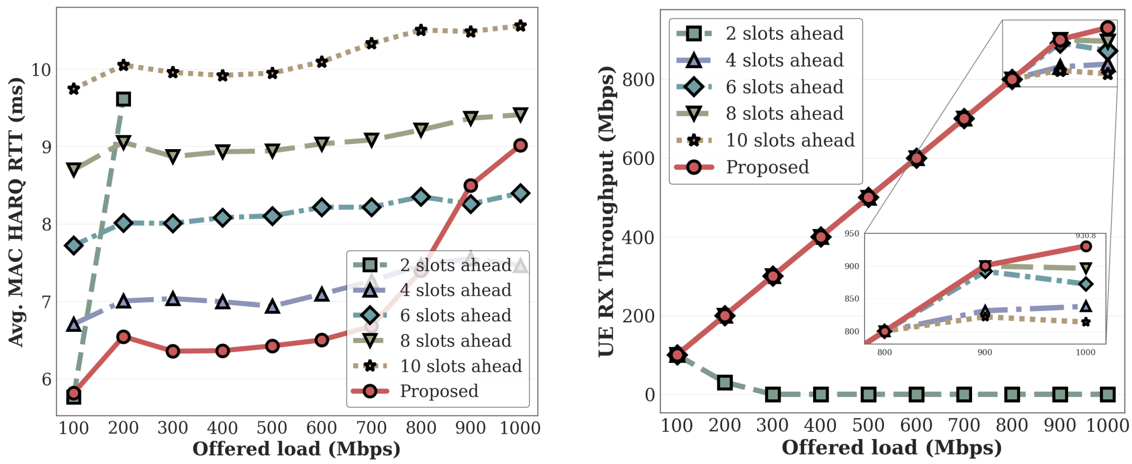
(a) Non-applied EWMA (b) Applied EWMA

Figure 1.2: Validation of EWMA processing on PNF Arrival Offset (Δt_{arrive}).

1.3 Scenario II: Performance Benchmark

In this scenario, we evaluate the baseline performance of the proposed adaptive timing mechanism compared to standard static slot-ahead configurations. Figure 1.3 presents the performance evaluation in terms of MAC layer latency and system throughput.

Figure 1.3a shows the MAC HARQ RTT under different offered loads. The proposed adaptive method maintains better Round-Trip Time (RTT) efficiency compared to static baselines while ensuring minimal packet loss. Figure 1.3b demonstrates the throughput optimization, confirming that the adaptive mechanism achieves the best throughput performance at peak loading, effectively eliminating the rare missing subframe errors caused by server OS preemption even in low-jitter environments.



(a) MAC Latency & Reliability

(b) Throughput Optimization

Figure 1.3: Performance evaluation: MAC RTT efficiency and system throughput.

1.4 Scenario III: Stability under Network Congestion

To test the robustness of the system, we injected artificial network congestion using Linux TC, varying the one-way delay (OWD) and jitter at the $T_2 - T_1$ and $T_5 - T_4$ segments. As depicted in Figure 1.4, while the static configuration experienced catastrophic throughput drops and frequent connection resets when latency exceeded the fixed window, the proposed adaptive timing control dynamically expanded the slot-ahead (S_{ahead}). This expansion allowed the MAC scheduler to pre-trigger messages, ensuring they arrived at the PNF within the valid window despite the increased transport delay. Consequently, the system maintained stable throughput even under congestion levels that previously caused complete synchronization failure.

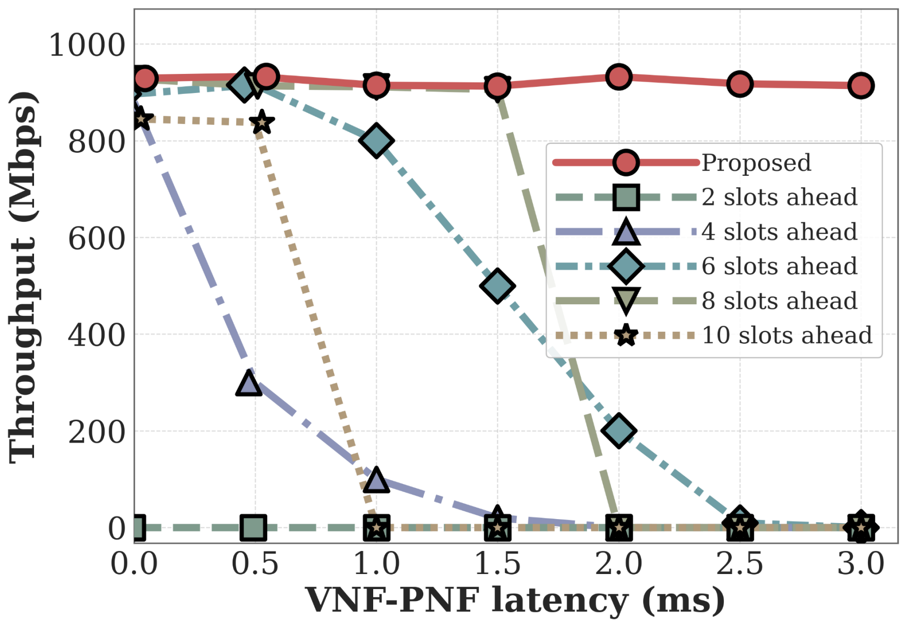


Figure 1.4: System stability and throughput performance under varying network congestion.

1.5 Functional Deployment Demonstration

To verify the end-to-end operational readiness of the proposed dual-loop timing control architecture, a functional demonstration was conducted within an SMO-integrated environment. The demonstration confirms the automated lifecycle management of the rApp, real-time latency tracking, and the dynamic synchronization of the VNF timing actuator in response to network fluctuations. This validates that the proposed system is fully functional and ready for deployment in realistic O-RAN scenarios.